کتابخانه‌ها می‌توانند مابین پردازه‌های متعددی به‌اشتراک گذاشته شوند، طوری که تنها یک نمونه از DLL در حافظه‌ی اصلی باشد. به‌همین دلیل، DLLها به **کتابخانه‌های مشترک** نیز معروف هستند و به‌صورت گسترده در سیستم‌های ویندوز و لینوکس استفاده می‌شوند.

موقعی که یک برنامه به یک روالی رجوع می‌کند که در یک کتابخانه‌ی پویاست، بارگذار DLL را پیدا کرده، در صورت لزوم به داخل حافظه بارگذاری می‌کند. در ادامه، آدرس‌ها را تطبیق می‌دهد تا توابع رجوع‌کننده به کتابخانه‌ی پویا به مکانی از حافظه که DLL ذخیره می‌شود ارجاع کنند.

کتابخانه‌های اتصالی پویا می‌توانند با به‌روزرسانی‌های کتابخانه (همچون ترمیم عیب‌ها[1]) گسترش پیدا کنند. به‌علاوه، کتابخانه می‌تواند با نسخه‌ی جدیدی جایگزین شود و تمامی برنامه‌هایی که به کتابخانه رجوع دارند به طور خودکار از این نسخه‌ی جدید استفاده نمایند. بدون داشتن اتصال پویا، تمامی این قبیل برنامه‌ها برای دستیابی کتابخانه‌ی جدید لازم دارند که اتصال خود را تجدید نمایند. لذا، برنامه‌ها به صورت تصادفی، نسخه‌های جدید ناسازگار کتابخانه‌ها را اجرا نخواهند کرد، چرا که اطلاعات مربوط به نسخه در هر دوی برنامه و کتابخانه نیز لحاظ می‌شود. حتی این امکان وجود دارد که نسخه‌های متعددی از کتابخانه‌ها در حافظه بارگذاری شوند و از این رو، هر یک از برنامه‌ها با استفاده از اطلاعات نسخه‌ی خود می‌توانند تصمیم بگیرند که از کدام نسخه استفاده نمایند. نسخه‌هایی که تغییرات اندکی دارند شماره‌ی نسخه‌ی خود را حفظ می‌کنند، حال آنکه با تغییرات اساسی، شماره‌ی نسخه افزایش می‌یابد. بنابراین، فقط برنامه‌هایی که با نسخه‌ی جدید کتابخانه کامپایل می‌شوند از تغییرات ناسازگار موجود آن متأثر خواهند شد. سایر برنامه‌هایی که اتصال آنها قبل از نصب کتابخانه‌ی جدید صورت گرفته است به استفاده از کتابخانه‌ی قدیمی‌تر ادامه می‌دهند.

برعکس بارگذاری پویا، اتصال پویا و کتابخانه‌های مشترک عموماً به کمک سیستم‌عامل نیاز دارند. اگر قرار باشد که پردازه‌های داخل حافظه از یکدیگر محافظت شوند، در این صورت سیستم‌عامل، تنها نهادی است که می‌تواند بررسی کند که ببیند آیا روال مورد نیاز در فضای حافظه‌ی پردازه‌ی دیگری قرار دارد و یا به پردازه‌های متعددی اجازه بدهد که آدرس‌های یکسانی از حافظه را دستیابی کنند. این مفهوم، همین‌طور چگونگی به اشتراک‌گذاری DLLها توسط پردازه‌های متعدد را با تمرکز بیشتری در بخش ۹-۳-۴، موقع بحث صفحه‌بندی بررسی می‌کنیم.

۹-۲ تخصیص همجوار حافظه

حافظه‌ی اصلی بایستی هر دوی سیستم‌عامل و پردازه‌های مختلف کاربر را تحت پوشش قرار بدهد. بنابراین، لازم داریم که بخش‌های حافظه‌ی اصلی را تا حد ممکن به شیوه‌ای کارآمد و مؤثر تخصیص بدهیم. این بخش، یک روش قدیمی، یعنی تخصیص همجوار حافظه[2] را تشریح می‌نماید.

1. Bug fixes
2. Contiguous memory allocation

حافظه معمولاً به دو پارتیشن تقسیم می‌شود: یکی برای سیستم‌عامل و دیگری برای پردازه‌های کاربر. می‌توانیم سیستم‌عامل را در آدرس‌های پایین حافظه و یا آدرس‌های بالای حافظه مستقر کنیم. این تصمیم به عوامل زیادی، همچون مکان بردار وقفه بستگی دارد. با این وجود، بسیاری از سیستم‌عامل‌ها (اعم از لینوکس و ویندوز)، سیستم‌عامل را در حافظه‌ی بالا قرار می‌دهند و لذا ما فقط این شرایط را بحث می‌کنیم.

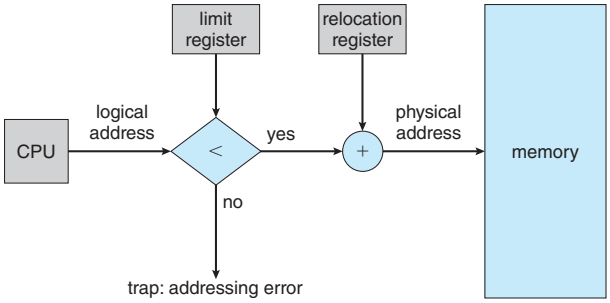
معمولاً می‌خواهیم پردازه‌های متعدد کاربر را همزمان در حافظه مستقر کنیم. بنابراین لازم است به این امر توجه داشته باشیم که حافظه‌ی موجود را چگونه به پردازه‌هایی که منتظر ورود به حافظه هستند تخصیص بدهیم. در **تخصیص همجوار حافظه**، هر پردازه در یک بخش منفرد از حافظه قرار می‌گیرد که با بخش بعدی که شامل پردازه‌ی دیگر است، همجوار است. هرچند، پیش از این که این طرح تخصیص حافظه را بیشتر بحث کنیم، بایستی به موضوع حفاظت حافظه بپردازیم.

۹-۲-۱ حفاظت حافظه

می‌توانیم با ترکیب دو اصلی که پیشتر عنوان شد، از دسترسی حافظه‌ای که یک پردازه مالکیت آن را ندارد ممانعت کنیم. اگر سیستمی با یک ثبات تغییر مکان (بخش ۹-۱-۳) به همراه یک ثبات حد (بخش ۹-۱-۱) داشته باشیم، به هدف خود می‌رسیم. ثبات تغییر مکان، کوچکترین مقدار آدرس فیزیکی را نگه می‌دارد؛ ثبات حد نیز بازه‌ی آدرس‌های منطقی را مشخص می‌کند (برای مثال، ۱۰۰۰۴۰ = تغییر مکان و ۷۴۶۰۰ = حد). هر آدرس منطقی بایستی در بازه‌ی مشخص شده توسط ثبات حد قرار بگیرد. MMU به صورت پویا با افزودن مقدار ثبات تغییر مکان، آدرس‌های منطقی را به حافظه نگاشت می‌دهد. این آدرس نگاشت خورده به حافظه ارسال می‌شود (شکل ۹-۶).

لحظه‌ای که زمانبند پردازنده پردازه‌ای را برای اجرا انتخاب می‌کند، اعزام‌کننده ثبات‌های تغییر مکان و حد را با مقادیر درست به عنوان بخشی از برگردان متن بارگذاری می‌کند. چون آدرس‌های تولیدی یک پردازنده با مقدار این ثبات‌ها مقایسه می‌گردد، می‌توانیم سیستم‌عامل و نیز سایر برنامه‌ها و داده‌های کاربران را از تغییرات این پردازه‌ی در حال اجرا محافظت کنیم.

طرح ثبات تغییر مکان، روش کارآمدی را برای تغییر پویای اندازه‌ی سیستم‌عامل فراهم می‌کند. این انعطاف‌پذیری در بسیاری از شرایط مطلوب است. برای مثال، سیستم‌عامل، فضای کد و بافر درایور دستگاه‌ها را نیز شامل می‌شود. اگر درایور دستگاهی فعلاً استفاده نداشته باشد، نگه‌داشتن آن در داخل حافظه دلیلی ندارد؛ می‌تواند تنها موقعی که نیاز است به داخل حافظه بارگذاری شود. همین‌طور، موقعی که درایور دستگاه دیگر نیاز نیست، می‌تواند برداشته شده و حافظه‌ی آن برای سایر نیازها تخصیص داده شود.

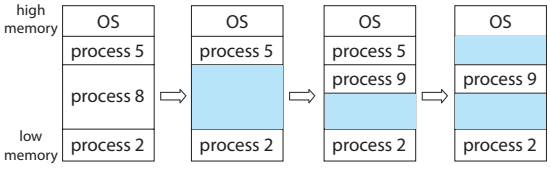


شکل ۹-۶ پشتیبانی سخت‌افزاری ثبات‌های تغییر مکان و حد.

۹-۲-۲ تخصیص حافظه

اکنون آمادگی داریم تا به تخصیص حافظه بپردازیم. یکی از ساده‌ترین روش‌های تخصیص حافظه، گماردن پردازه‌ها به پارتیشن‌های با اندازه‌ی متغیر در داخل حافظه است که هر پارتیشن می‌تواند دقیقاً یک پردازه را نگه بدارد. در این طرح **پارتیشن متغیر**، سیستم‌عامل یک جدولی نگه می‌دارد که نشان می‌دهد کدام بخش‌های حافظه در دسترس بوده و کدام بخش‌ها اشغال شده‌اند. در آغاز، تمامی حافظه در دسترس پردازه‌های کاربر قرار دارد و به مانند یک بلاک بزرگ حافظه، مثل یک **حفره** می‌باشد. سرانجام، چنان که خواهید دید، حافظه مجموعه‌ای از حفره‌ها با اندازه‌های مختلف را شامل می‌شود.

شکل ۹-۷، این طرح را به‌تصویر می‌کشد. در آغاز، حافظه به‌صورت کامل مورد استفاده قرار می‌گیرد که شامل پردازه‌های ۵، ۸، و ۲ می‌باشد. بعد از این که پردازه‌ی ۸ خارج می‌شود، یک حفره‌ی همجوار وجود دارد. بعداً، پردازه‌ی ۹ می‌رسد و حافظه می‌گیرد. در ادامه، پردازه‌ی ۵ حافظه را ترک می‌کند که نتیجه، دو حفره‌ی غیرهمجوار است.



شکل ۹-۷ پارتیشن متغیر.

همچنان که پردازه‌ها به سیستم وارد می‌شوند، سیستم‌عامل جهت تعیین این که، حافظه را به کدام یک از پردازه‌ها تخصیص خواهد داد، نیازهای هر یک از پردازه‌ها و حجم فضای موجود حافظه را بررسی می‌کند. زمانی که به یک پردازه، فضا تخصیص یافت، داخل حافظه بارگذاری شده که در ادامه می‌تواند برای کسب زمان پردازنده به رقابت بپردازد. زمانی که یک پردازه خاتمه می‌پذیرد، حافظه‌اش را رها می‌سازد که پس از آن سیستم‌عامل می‌تواند به پردازه‌ی دیگری تخصیص بدهد.

چه اتفاقی می‌افتد در صورتی که برای ارضای تقاضاهای یک پردازه‌ی رسیده حافظه‌ی کافی فراهم نباشد؟ یک انتخاب این است که به‌سادگی پردازه را رد کرده و یک پیغام خطای مناسبی

داده شود. به روشی دیگر، می‌توانیم این قبیل پردازه‌ها را در یک صف انتظار بگذاریم. سیستم‌عامل، موقعی که بعداً حافظه آزاد می‌شود، صف انتظار را بررسی می‌کند تا تعیین کند آیا می‌تواند تقاضاهای حافظه‌ی یک پردازه‌ی منتظر را ارضا نماید یا خیر.

در حالت کلی، همچنان که عنوان شد، بلاک‌های در دسترس حافظه مرکب از مجموعه‌ای از حفره‌ها با اندازه‌های مختلف هستند که در کل حافظه پراکنده شده‌اند. زمانی که پردازه‌ای می‌رسد و نیاز به حافظه دارد، سیستم این مجموعه را می‌گردد تا حفره‌ای به اندازه‌ی کافی بزرگ را برای این پردازه بیابد. اگر حفره زیادی بزرگ باشد، به دو قسمت شکسته می‌شود. یک بخش آن به پردازه‌ی آمده تخصیص می‌یابد؛ قسمت دیگر آن به مجموعه‌ی حفره‌ها برگردانده می‌شود. زمانی هم که پردازه خاتمه یافت، بلاک حافظه‌اش را رها می‌سازد تا دوباره در مجموعه‌ی حفره‌ها گذاشته شود. اگر حفره‌ی جدید با حفره‌های دیگری مجاور باشد، این حفره‌های مجاور هم با همدیگر ادغام می‌شوند تا حفره‌ای بزرگ‌تر را تشکیل بدهند.

این روال، نمونه‌ی به‌خصوصی از **مسأله‌ی تخصیص پویای حافظه‌ی**[1] کلی است که به چگونگی ارضای درخواستی با اندازه‌ی n از لیست حفره‌های آزاد مربوط می‌شود. راه‌حل‌های بسیاری برای این مسأله وجود دارد. راهبردهای **اولین-گنجایش**[2]، **بهترین-گنجایش**[3] و **بدترین-گنجایش**[4] رایج‌ترین نمونه‌هایی هستند که برای انتخاب یک حفره‌ی آزاد از مجموعه‌ی حفره‌های موجود استفاده می‌شوند.

- **اولین‌گنجایش**: اولین حفره‌ای را تخصیص بدهید که به اندازه‌ی کافی بزرگ باشد. جستجو می‌تواند از ابتدای مجموعه‌ی حفره‌های موجود و یا از مکان باقیمانده از جستجوی قبلی شروع شود. به محض این که حفره‌ی آزادی یافتیم که به اندازه‌ی کافی بزرگ بود، می‌توانیم جستجو را متوقف کنیم.
- **بهترین‌گنجایش**: کوچکترین حفره‌ای را تخصیص بدهید که به اندازه‌ی کافی بزرگ است. بایستی کل لیست را جستجو کنیم، مگر این که لیست بر اساس اندازه مرتب شده باشد. این راهبرد، کوچکترین حفره را باقی می‌گذارد.
- **بدترین‌گنجایش**: بزرگ‌ترین حفره را تخصیص بدهید. در اینجا نیز بایستی کل لیست را بگردیم، مگر این که بر اساس اندازه مرتب شده باشد. این راهبرد، بزرگ‌ترین حفره را باقی می‌گذارد که می‌تواند مفیدتر از کوچکترین حفره‌ای باشد که از روش بهترین‌گنجایش باقی می‌ماند.

شبیه‌سازی‌ها نشان داده‌اند که هر دو روش اولین گنجایش و بهترین گنجایش با تعابیر کاهش زمان و سودمندی حافظه از بدترین گنجایش بهتر هستند. از نظر بهره‌وری حافظه چندان روشن نیست که اولین‌گنجایش و یا بهترین‌گنجایش از سایر راهبردها بهتر خواهند بود، ولی اولین‌گنجایش عموماً سریع‌تر است.

1. Dynamic storage allocation problem
2. First-fit
3. Best-fit
4. Worst-fit

۹-۲-۳ پارگی

راهبردهای تخصیص حافظه‌ی اولین-گنجایش و بهترین-گنجایش، هر دو از **پارگی خارجی**[1] رنج می‌برند. همچنان که پردازه‌ها بارگذاری شده و از حافظه حذف می‌شوند، فضای آزاد حافظه به تکه‌های کوچکی شکسته می‌شود. پارگی خارجی زمانی به وجود می‌آید که فضای حافظه در مجموع جوابگوی یک درخواست می‌باشد، اما فضاهای آزاد موجود همجوار نبوده و حافظه به تعداد زیادی از حفره‌های کوچک تکه‌تکه می‌شود. مسأله‌ی پارگی می‌تواند جدی باشد. در بدترین حالت، میان هر دو پردازه یک بلاک آزاد حافظه (یا هدر رفته) خواهیم داشت. در عوض، اگر تمامی این تکه‌های کوچک حافظه در یک بلاک آزاد بزرگی می‌بودند می‌توانستیم پردازه‌های بیشتری را به اجرا در آوریم.

این که از کدام یک از راهبردهای اولین-گنجایش و یا بهترین-گنجایش استفاده کنیم، می‌تواند روی میزان پارگی تأثیرگذار باشد (در برخی سیستم‌ها، اولین‌گنجایش بهتر عمل می‌کند، حال آنکه برای برخی سیستم‌های دیگر، بهترین‌گنجایش مناسب است). عامل دیگر این است که بلاک آزاد از کدام طرف تخصیص می‌یابد (یعنی، تکه‌ای که باقی می‌ماند در بالای حفره می‌باشد یا ته آن؟). با این وجود، فارغ از الگوریتمی که استفاده می‌شود، پارگی خارجی می‌تواند یک مسأله باشد.

پارگی خارجی بسته به حجم کل ذخیره‌ی حافظه و اندازه‌ی متوسط پردازه می‌تواند به عنوان یک مسأله‌ی اصلی یا فرعی مطرح شود. برای نمونه، تحلیل آماری اولین‌گنجایش حاکی از آن است که حتی به رغم برخی بهینه‌سازی‌ها، با داشتن N بلاک تخصیص‌یافته، 0.5N بلاک دیگر به خاطر پارگی به هدر خواهد رفت. یعنی، یک سوم حافظه می‌تواند بلااستفاده بماند! این ویژگی به **قانون ۵۰ درصد** معروف است.

پارگی حافظه در کنار خارجی می‌تواند داخلی هم باشد. طرح تخصیص چندپارتیشنی را با حفره‌ی ۱۸۴۶۴ بایتی در نظر بگیرید. فرض کنید پردازه‌ی بعدی ۱۸۴۶۲ بایت درخواست می‌کند. اگر بلاک درخواستی پردازه را دقیقاً تخصیص بدهیم، حفره‌ای به اندازه‌ی ۲ بایت برای ما باقی می‌ماند. سربار حفظ رد این حفره اساساً از خود حفره بیشتر خواهد بود. روش کلی اجتناب از این مسأله آن است که حافظه‌ی فیزیکی را به بلاک‌هایی با اندازه‌های ثابت بشکنیم و حافظه را بر اساس واحدهایی از اندازه‌ی بلاک تخصیص بدهیم. با این روش، حافظه‌ای که به یک پردازه تخصیص می‌یابد می‌تواند کمی بزرگ‌تر از حافظه‌ی درخواستی آن باشد. اختلاف میان این دو عدد، **پارگی داخلی** (حافظه‌ی بلااستفاده‌ای که در داخل یک پارتیشن قرار می‌گیرد) است.

یک راه‌حل مسأله‌ی پارگی خارجی **فشرده‌سازی** است. هدف آن است که محتویات حافظه طوری جابجا شوند تا تمامی فضاهای آزاد حافظه در یک بلاک بزرگ در کنار هم قرار بگیرند. با این وجود، فشرده‌سازی هم همیشه امکان‌پذیر نیست. اگر تغییر مکان ایستا باشد و در زمان اسمبل یا بارگذاری انجام شود، فشرده‌سازی امکان‌پذیر نیست. آن فقط در صورتی امکان‌پذیر است که تغییر مکان، پویا و در زمان اجرا انجام شود. اگر آدرس‌ها به صورت پویا تغییر مکان یابند، برای تغییر مکان تنها لازم است که برنامه و داده‌های آن جابجا شده و سپس ثبات مبنا

1. External fragmentation

تغییر یابد تا آدرس مبنای جدید را منعکس نماید. زمانی که فشرده‌سازی امکان‌پذیر است، بایستی هزینه‌ی آن تعیین شود. ساده‌ترین الگوریتم فشرده‌سازی آن است که تمامی پردازه‌ها به یک طرف حافظه و تمامی حفره‌ها به طرف دیگر حافظه جابجا شوند که با این کار، حفره‌ی بزرگی از حافظه فراهم می‌شود. این طرح می‌تواند گران تمام شود.

راه‌حل ممکن دیگر مسأله‌ی پارگی خارجی آن است که اجازه بدهیم فضای آدرس منطقی پردازه‌ها غیرهمجوار گردد، لذا پردازه بتواند فضای فیزیکی حافظه را در هر جا که شد در اختیار بگیرد. این راهبرد مورد استفاده در *صفحه‌بندی*، رایج‌ترین تکنیک مدیریت حافظه در سیستم‌های کامپیوتری است. ما صفحه‌بندی را در بخش بعدی بیان می‌کنیم.

پارگی یک مسأله‌ی عمومی در محاسبات است که می‌تواند در هر جایی که می‌بایست بلاک‌هایی از داده‌ها را مدیریت کنیم، اتفاق بیفتند. این موضوع را با عمق بیشتر در فصول مدیریت ذخیره‌سازی (فصل ۱۱ تا فصل ۱۵) بحث و بررسی می‌کنیم.

۹-۳ صفحه‌بندی

مدیریت حافظه‌ای که تاکنون بحث شده لازم دارد که فضای آدرس فیزیکی یک پردازه همجوار باشد. اکنون **صفحه‌بندی** را معرفی می‌کنیم، یک طرح مدیریت حافظه‌ای که اجازه می‌دهد فضای آدرس فیزیکی یک پردازه غیرهمجوار گردد. صفحه‌بندی از پارگی خارجی و نیاز به فشرده‌سازی مرتبط اجتناب می‌کند، دو مسئله‌ای که تخصیص همجوار حافظه با آن درگیر است. صفحه‌بندی به‌خاطر مزایای متعددی که دارد در اَشکال گوناگونش در اکثر سیستم‌عامل‌ها، از کارگزارهای بزرگ گرفته تا دستگاه‌های موبایل مورد استفاده قرار می‌گیرد. صفحه‌بندی با همکاری میان سیستم‌عامل و سخت‌افزار کامپیوتر پیاده‌سازی می‌شود.

۹-۳-۱ روش پایه‌ای

روش پایه‌ای پیاده‌سازی صفحه‌بندی، شکستن حافظه‌ی فیزیکی به بلاک‌هایی با اندازه‌ی ثابت به نام **قاب‌ها**[1] و شکستن حافظه‌ی منطقی به بلاک‌هایی با اندازه‌ی یکسان به نام **صفحات** می‌باشد. زمانی که پردازه‌ای اجرا می‌شود، صفحات آن از ذخیره‌ی پشتیبان در داخل هر یک از قاب‌های موجود حافظه بارگذاری می‌گردند. ذخیره‌ی پشتیبان به بلاک‌هایی با اندازه‌ی ثابت تقسیم می‌شود که با قاب‌های حافظه یا خوشه‌هایی از قاب‌های متعدد، هم اندازه هستند. این ایده‌ی به ظاهر ساده کارکردی فوق‌العاده و دسته‌های وسیعی دارد. برای مثال، هم‌اکنون فضای آدرس منطقی به صورت کامل از فضای آدرس فیزیکی جدا شده است، طوری که یک پردازه می‌تواند یک فضای آدرس منطقی ۶۴ بیتی، حتی در سیستمی داشته باشد که کمتر از 2^{64} بایت حافظه‌ی فیزیکی دارد.

هر یک از آدرس‌های تولید شده توسط پردازنده به دو قسمت تقسیم می‌شوند: **شماره‌ی صفحه (p)** و **افست صفحه (d)**.

1. Frames